Using Anthropic's **Claude Code** for Research

Joe Hahn, Space Science Institute

SSI AI Working Group, Oct 28, 2026

- Claude Code (CC) is an AI *agent* that works inside my project folder
  - it reads my files, writes & runs code, and I approve each step
  - not a chat window
- context is everything:
  - CC reads my code, notes, papers & data before it answers, so its outputs align with the project's goals
  - it also manages all new code development, the paper & this deck, and will eventually draft my next funding proposal
- I'm the advisor, CC is my grad student:
  - CC is very good at math, code & data analysis
  - but needs direction: derive it this way not that way and test, test, test
  - like a student it can drift, and rarely says "I don't know"

a CC exchange from this project, condensed:

```
> should "Δ² = d_r² + d_z² - c(d_r s_r + d_z s_z)" be written
delta^2=dr^2+dz^2-(cN)^2 instead?
● Yes: they are equal, and the new form is just Pythagoras, |d0|² = Δ² + |d1|².
● edits Eqn. 21 in the paper
● writes sympy code that re-derives Δ² from the geometry, runs it:
  PASS G16: Δ² = d_r² + d_z² - (Nc)²
● recompiles the paper
```

paper Eqn. (21), before:

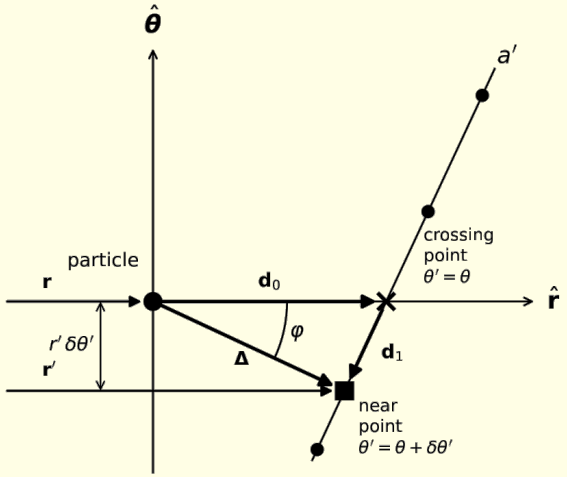
$$\boldsymbol{\Delta} = (d_r - cs_r)\hat{\mathbf{r}} - c\hat{\boldsymbol{\theta}} + (d_z - cs_z)\hat{\mathbf{z}}$$
$$\Delta^2 = d_r^2 + d_z^2 - c(d_r s_r + d_z s_z). \qquad (21)$$

after:

$$\boldsymbol{\Delta} = (d_r - cs_r)\hat{\mathbf{r}} - c\hat{\boldsymbol{\theta}} + (d_z - cs_z)\hat{\mathbf{z}}$$
$$\Delta^2 = d_r^2 + d_z^2 - (Nc)^2. \qquad (21)$$

Literature search & citations:

- CC reads the papers in my project folder & finds what I need
  - eg the wire-gravity geometry in Fig. 3 of Hahn & Spitale (2013), which CC redrew & extended to 3D



- but AI can invent citations
  - CC confirms each citation's authors, year, journal, volume & page in NASA ADS
- what CC knows from training vs what it reads in my folder:
  - from training: well-known results & widely cited papers,
    eg the viscous overstability papers it cited when we chased a numerical instability,
    none of which were in my folder; all four checked out in ADS
  - what's in my project folder: my papers, code & notation, code output, notes,
    all the specifics that CC will need to advance the research
  - training knowledge is broad but fuzzy on details,
    so I copy the key papers into my project folder, where CC reads them

Physics & equations of motion:

- who does what:
  - me: I set up the problem to be solved
  - CC: then does the physical & mathematical reasoning
  - sympy (Python symbolic math): CC calls sympy, which re-derives each equation from its definitions & confirms the result
  - me: I check the equations by hand & suggest alternative derivations
- eg I asked CC: "I suspect $d_1 \ll \Delta$, try to derive a simple proof"
  - CC reasoned it through: $|d_1| \le s\Delta$, where $s \ll 1$ is the streamline's slope
  - CC then put the proof in the paper & added sympy checks
- CC also reviewed my predecessor code *epi_int_lite*: it found a few small bugs and two approximations worth improving upon

Eqn. 7: solved via recall (a textbook result)

ing $\nu^2(r)$ at $r = a$ reduces the vertical equation to the harmonic oscillator

$$\ddot{z} = -\nu_a^2 z, \tag{7}$$

with vertical frequency (Eqn. 17 of N. Borderies-Rappaport & P.-Y. Longaretti 1994)

$$\nu_a \equiv \nu(a) = \sqrt{\frac{GM_p}{a^3}}\left[1 + \frac{9}{2}J_2\left(\frac{R_p}{a}\right)^2\right]^{1/2}, \tag{8}$$

Eqns. 19–21: stepwise reasoning about the geometry (figure on previous page)

The near point is where $\boldsymbol{\Delta}$ is perpendicular to the streamline, $\boldsymbol{\Delta} \cdot \hat{\mathbf{t}} = (\mathbf{d}_0 + \mathbf{d}_1) \cdot \hat{\mathbf{t}} = 0$. And because $\mathbf{d}_1$ lies along $\hat{\mathbf{t}}$, this requires $\mathbf{d}_1 = -(\mathbf{d}_0 \cdot \hat{\mathbf{t}})\hat{\mathbf{t}}$, so

$$\boldsymbol{\Delta} = \mathbf{d}_0 + \mathbf{d}_1 = \mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{t}})\hat{\mathbf{t}}, \tag{19}$$

which is simply $\mathbf{d}_0$ with its component along the streamline removed.

Now equate $\mathbf{d}_1 = -(\mathbf{d}_0 \cdot \hat{\mathbf{t}})\hat{\mathbf{t}}$ to Eqn. (16), which shows that the near point lies at $\delta\theta' = -c/r'$, where $c = (\mathbf{d}_0 \cdot \hat{\mathbf{t}})/N = (d_r s_r + d_z s_z)/N^2$. So at the near point

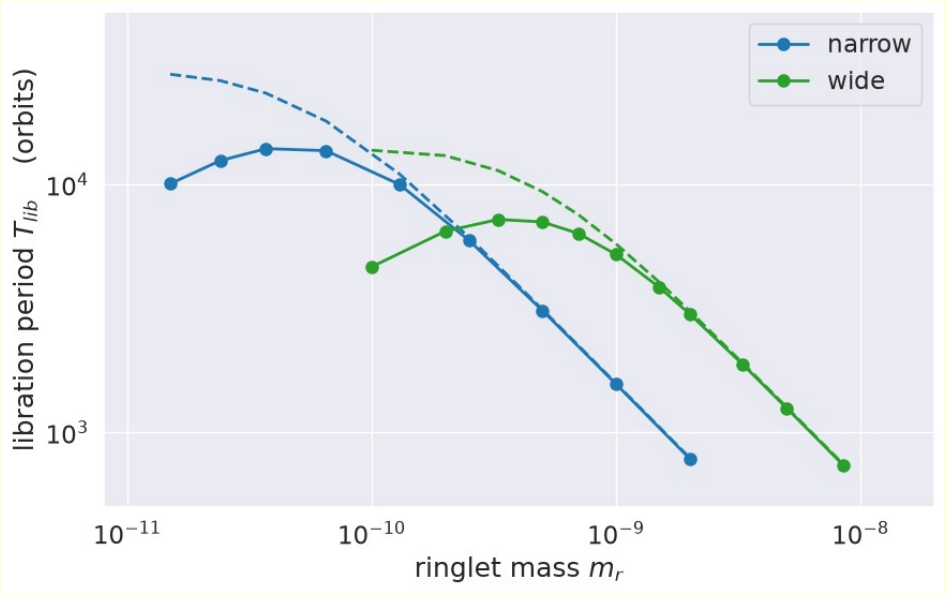
$$\mathbf{d}_1 = -Nc\,\hat{\mathbf{t}} = -c(s_r\hat{\mathbf{r}} + \hat{\boldsymbol{\theta}} + s_z\hat{\mathbf{z}}). \tag{20}$$

Adding $\mathbf{d}_0$ to the above then yields $\boldsymbol{\Delta}$:

$$\boldsymbol{\Delta} = (d_r - cs_r)\hat{\mathbf{r}} - c\hat{\boldsymbol{\theta}} + (d_z - cs_z)\hat{\mathbf{z}}$$
$$\Delta^2 = d_r^2 + d_z^2 - (Nc)^2. \tag{21}$$

CC manages numerical simulations, tested against theory:

- CC ported my old Python 2 code to Python 3.14, then made it ~35× faster
  - using *Numba*, which compiles Python like Fortran or C & parallelizes it across all CPU cores; identical results
- every test simulation is compared to theory
  - viscous spreading, libration rates, nodal regression
- regression tests flag any change in the answers, to 1 part in 10^{11}



simulated ringlets (dots) vs Borderies, Goldreich & Tremaine theory (dashed)

- trust in the derivations & code is earned by comparing simulations to well-understood theory

The near point is where $\mathbf{\Delta}$ is perpendicular to the streamline, $\mathbf{\Delta}\cdot\hat{\mathbf{t}} = (\mathbf{d}_0+\mathbf{d}_1)\cdot\hat{\mathbf{t}} = 0$. And because $\mathbf{d}_1$ lies along $\hat{\mathbf{t}}$, this requires $\mathbf{d}_1 = -(\mathbf{d}_0\cdot\hat{\mathbf{t}})\hat{\mathbf{t}}$, so

$$\mathbf{\Delta} = \mathbf{d}_0 + \mathbf{d}_1 = \mathbf{d}_0 - (\mathbf{d}_0\cdot\hat{\mathbf{t}})\hat{\mathbf{t}}, \tag{19}$$

which is simply $\mathbf{d}_0$ with its component along the streamline removed.

Now equate $\mathbf{d}_1 = -(\mathbf{d}_0\cdot\hat{\mathbf{t}})\hat{\mathbf{t}}$ to Eqn. (16), which shows that the near point lies at $\delta\theta' = -c/r'$, where $c = (\mathbf{d}_0\cdot\hat{\mathbf{t}})/N = (d_r s_r + d_z s_z)/N^2$. So at the near point

$$\mathbf{d}_1 = -Nc\,\hat{\mathbf{t}} = -c(s_r\hat{\mathbf{r}} + \hat{\boldsymbol{\theta}} + s_z\hat{\mathbf{z}}). \tag{20}$$

Adding $\mathbf{d}_0$ to the above then yields $\mathbf{\Delta}$:

$$\begin{aligned}\mathbf{\Delta} &= (d_r - cs_r)\hat{\mathbf{r}} - c\hat{\boldsymbol{\theta}} + (d_z - cs_z)\hat{\mathbf{z}}\\ \Delta^2 &= d_r^2 + d_z^2 - (Nc)^2.\end{aligned} \tag{21}$$

Inserting $\mathbf{\Delta}$ into Eqn. (13) then provides the acceleration that streamline a' exerts on the particle:

$$\begin{aligned}A_{g,r} &= \frac{2G\lambda'(d_r - cs_r)}{\Delta^2}\\ A_{g,\theta} &= -\frac{2G\lambda' c}{\Delta^2}\\ A_{g,z} &= \frac{2G\lambda'(d_z - cs_z)}{\Delta^2},\end{aligned} \tag{22}$$

where $N^2 = 1 + s_r^2 + s_z^2$ and $c = (d_r s_r + d_z s_z)/N^2$. Note that the near point is much closer than Δ to the crossing point: Eqns. (20)–(21) imply $|\mathbf{d}_1| \le s\Delta$, where $s = (s_r^2 + s_z^2)^{1/2}$, so for the small slopes of a nearly circular and nearly planar streamline $|\mathbf{d}_1| \ll \Delta$.

3.2. *Computing the acceleration*

The particles' coordinates are stored in arrays whose rows are the streamlines, and every timestep, before the

Writing the science:

- CC manages the LaTeX/AASTeX, bibliography & the code that makes the figures
- CC drafted a style guide from my last two papers, refined it with my edits,
  & writes in my voice
- I edit by pasting text from the PDF, eg "shorten …"
  - before: …so the particles that bracket the perturbed particle's longitude θ are easily identified: they are that particle and its neighbor on either side.
  - after: …so it and its neighbor on either side bracket θ.
- my judgment, my voice; its typing
- advice: start the research project by writing a project plan, verify everything along the way, and save every prompt & change

About me:

- **Joe Hahn**, planetary scientist at the Space Science Institute
  - research interests: planetary rings, planetary dynamics & the zodiacal cloud; planet formation & circumstellar disks; computational dynamics
- also an AI consultant & builder at **JMH DataSciences**
  - 12+ years building production AI & machine learning, eight at Oracle, with the last three in its AI Center of Excellence
  - I now run my own independent AI consulting shop:
    - I build AI tools that do work a machine should do
    - & provide senior hands to AI teams that have more work than people
- this project's code, draft paper & every prompt I gave CC:
  - github.com/joehahn/epi_int_3d

jmh-datasciences.com joe.hahn@jmh-datasciences.com